

Digital Egypt Pioneers Initiative

DEPI

Graduation Project Documentation

Telbs Eh

تلبس ايه ؟

A Weather-Driven Outfit Recommendation System
Powered by an End-to-End Data Engineering Pipeline

Project Title	Telbs Eh — Weather-Based Outfit Recommendation System
Document Type	Project Documentation
Domain	Data Engineering • ETL • Data Warehousing • Application Layer
Cloud Platform	Microsoft Azure
Institution	Digital Egypt Pioneers Initiative (DEPI)
Academic Supervisor	Dr. [Supervisor Name]
Academic Year	2025 / 2026

Project Team

#	Team Member
1	Mohamed Hesham Mohamed
2	Maram Ragab Saady
3	Habiba Ashraf El-Badawy
4	Youssef Yahia
5	Mostafa Mahmoud
6	Faten Reyad Mahrous

Table of Contents

Table of Contents.....	3
1. Executive Summary.....	5
1.1 Project at a Glance	5
2. Project Planning & Management.....	6
2.1 Project Vision	6
2.2 Project Objectives	6
2.3 Project Scope	6
In Scope.....	6
Out of Scope.....	6
2.4 Project Team & Roles.....	7
2.5 Project Timeline & Phases	7
2.6 Tools & Technology Stack	7
2.7 Risk Management	7
3. Literature Review	9
3.1 Weather Data Sources & APIs.....	9
3.2 ETL Concepts & Best Practices	9
3.3 Data Warehousing — Star vs Snowflake	10
3.4 Workflow Orchestration with Airflow	10
3.5 Recommendation Systems Overview	10
3.6 Comparable Solutions	11
4. Requirements Gathering.....	12
4.1 Stakeholders	12
4.2 Functional Requirements.....	12
4.3 Non-Functional Requirements.....	13
4.4 Data Requirements	13
4.5 Use Cases (Summary).....	14
4.6 Detailed Use Case Diagram	14
4.6.1 Actors Description.....	14
4.6.2 Detailed Use Case Specifications	15
UC-04 — Get Outfit Recommendation	15
UC-06 — Refresh Current Weather (ETL)	15

UC-08 — Reconcile City Names & Coordinates	16
4.6.3 Use Case Relationships Summary	17
5. System Analysis & Design	18
5.1 High-Level Architecture	18
5.1.1 Architecture Diagram (Logical Flow).....	18
5.2 Data Acquisition Strategy.....	19
5.2.1 Two Real-World Data Issues Encountered	19
5.2.2 Three-File Resolution Approach	19
5.3 ETL Pipeline Design	19
5.3.1 Extract	20
5.3.2 Transform.....	20
5.3.3 Load.....	20
5.4 OLTP Schema (DDL Layer)	20
5.5 Application Layer — Recommendation Engine	21
5.6 Data Warehouse — Snowflake Schema (Azure).....	21
5.6.1 Fact Table	21
5.6.2 Dimension Tables (Snowflaked).....	21
5.6.3 Sample Analytical Questions Answered by the DW	22
5.7 Orchestration with Apache Airflow	22
5.8 Deployment on Microsoft Azure	22
5.9 Current Status & Next Steps	23
6. Conclusion	24

1. Executive Summary

Telbs Eh (تلبس إيه؟) is a smart, weather-driven outfit recommendation system targeting users across Egypt. The application answers a simple but recurring daily question: "What should I wear today?" — by combining real-time and forecast weather data with a structured recommendation engine that maps weather attributes (temperature, humidity, wind, precipitation, weather condition) to suitable clothing categories.

The project is built as a complete end-to-end data engineering solution. Weather data is collected from the OpenWeather API, transformed through a Python-based ETL pipeline, persisted in an OLTP database to serve the application layer, and then loaded into an analytical Data Warehouse on Microsoft Azure for historical and statistical analysis. Workflow orchestration is handled by Apache Airflow, while the warehouse layer is modeled as a Snowflake Schema.

This document describes the project planning approach, the literature reviewed, the gathered functional and non-functional requirements, and the system analysis and design — including data flows, schema design, and architectural decisions.

1.1 Project at a Glance

1. Actor: End User.
2. Goal: receive a clothing recommendation based on current weather in the selected city.
3. Flow: user selects a city → system fetches latest weather from OLTP → recommendation engine evaluates temperature, humidity, wind, and condition → outfit category is returned and logged.
4. If no weather data exists for the city, the system shows a data unavailable message.

2. Project Planning & Management

2.1 Project Vision

Telbs Eh aims to bridge a small but persistent everyday gap: people frequently misjudge weather and end up dressed inappropriately for the day. By integrating reliable weather data with a recommendation engine, the application removes guesswork and offers context-aware suggestions tailored to the user's city in Egypt.

2.2 Project Objectives

- Build a reliable ETL pipeline that ingests weather data from a public API for all Egyptian regions.
- Resolve real-world data issues: missing coordinates and inconsistent location naming between source and internal references.
- Persist transactional data in an OLTP layer to serve the application in real time.
- Provide outfit recommendations through an application layer driven by weather attributes.
- Build an analytical Data Warehouse (Snowflake Schema) on Microsoft Azure for historical and statistical reporting.
- Orchestrate the full pipeline using Apache Airflow with scheduled, monitored DAGs.

2.3 Project Scope

In Scope

- Data ingestion from OpenWeather API (current + forecast).
- Data cleaning, mapping, and reconciliation of city names and coordinates.
- OLTP DDL design and population for application use.
- Application layer that recommends outfits based on weather attributes.
- Snowflake Schema Data Warehouse on Azure for analytics.
- Airflow-orchestrated, scheduled ETL workflows.

Out of Scope

- Mobile-native deployment to app stores (initial version focuses on the data + recommendation backbone).

- Personalization beyond weather attributes (e.g. style preferences, brand integration).
- Real-time streaming ingestion (the pipeline is batch-scheduled).

2.4 Project Team & Roles

The team consists of six members. Roles overlap across multiple stages of the pipeline to ensure shared knowledge and proper code review across the data engineering, application, and warehouse tracks.

5. Actor: System Scheduler (Airflow).
6. Goal: keep the OLTP database up to date with the latest weather for all Egyptian cities.
7. Flow: Airflow triggers `current_weather_dag` every 2 hours → reads city list → calls OpenWeather API → loads DWH → truncates OLTP → inserts fresh data → updates `dim_date`.
8. On API failure: task retries up to 3 times. On load failure: DAG run is marked as failed.

2.5 Project Timeline & Phases

The project follows an iterative phased approach, with each phase producing a tangible deliverable that feeds into the next.

9. Actor: System Scheduler — invoked automatically during every ETL run.
10. Goal: resolve mismatches between OpenWeather API city names and canonical Egyptian names, and fill missing coordinates from a manual file.
11. Flow: load master city list → load manual lat/lon file → apply name-mapping dictionary → merge on city ID → fill null coordinates → output a clean unified DataFrame.
12. Any city still missing coordinates after the merge is excluded from the current run and flagged for manual review.

2.6 Tools & Technology Stack

13. 1. OpenWeather API — data source via HTTPS/JSON (current weather + 5-day forecast).
14. 2. ETL Layer (Python + Pandas) — extracts from API, cleans data, reconciles city names and coordinates.
15. 3. OLTP Database (MySQL: `telbs_ah_final_proj`) — operational snapshot store, truncated and reloaded on each run.
16. 4. Application Layer (Streamlit) — reads from OLTP, displays weather and outfit recommendations.
17. 5. DWH Load — before each OLTP truncate, data is saved permanently to the DWH fact tables.
18. 6. Data Warehouse (MySQL: `telbs_ah_dwh`) — historical store with `dim_city`, `dim_date`, `fact_weather_observation`, `fact_weather_forecast`.

2.7 Risk Management

19. 1. OpenWeather API — data source via HTTPS/JSON (current weather + 5-day forecast).
20. 2. ETL Layer (Python + Pandas) — extracts from API, cleans data, reconciles city names and coordinates.
21. 3. OLTP Database (MySQL: telbs_eh_final_proj) — operational snapshot store, truncated and reloaded on each run.
22. 4. Application Layer (Streamlit) — reads from OLTP, displays weather and outfit recommendations.
23. 5. DWH Load — before each OLTP truncate, data is saved permanently to the DWH fact tables.
24. 6. Data Warehouse (MySQL: telbs_eh_dwh) — historical store: dim_city, dim_date, fact_weather_observation, fact_weather_forecast.

3. Literature Review

Before designing the system, the team reviewed a body of work covering weather APIs, ETL design patterns, data warehousing methodologies, and recommendation systems. The review focused on identifying proven approaches and highlighting how Telbs Eh fits within (and slightly extends) existing solutions.

3.1 Weather Data Sources & APIs

Multiple public weather APIs exist (OpenWeather, WeatherAPI, AccuWeather, Tomorrow.io). They differ in coverage, rate limits, accuracy, and data fields. OpenWeather was selected for the project for the following reasons:

- Generous free tier suitable for an academic project.
- Global coverage that includes most Egyptian cities.
- Multiple endpoints: current weather, hourly/daily forecast, and geocoding.
- Well-documented JSON responses, consumable directly with Python's requests library.

However, the team observed during integration that the API does not perfectly match Egyptian administrative naming conventions: some governorates appear under their English literal translations (e.g. "البحيرة" — Beheira — is sometimes returned as "The Lake"), and some smaller regions lack accurate latitude/longitude entries. These observations directly shaped the ETL design described in Section 5.

3.2 ETL Concepts & Best Practices

ETL (Extract — Transform — Load) is the canonical pattern for moving data between operational and analytical systems. Key principles drawn from the literature:

- Idempotency — re-running an ETL job should not corrupt or duplicate data.
- Separation of concerns — each step (extract, transform, load) should be independently testable.
- Data quality first — invalid records should be quarantined and logged, not silently dropped.
- Schema-on-write for OLTP, with strict typing; tolerant ingestion at the staging boundary.

Pandas is the chosen transformation engine because of its concise API for joins, type coercion, null handling, and merging — all of which are required heavily by Telbs Eh due to the city-name reconciliation problem.

3.3 Data Warehousing — Star vs Snowflake

Two dominant dimensional modeling approaches dominate analytical design:

- **Star Schema:** denormalized dimensions, simpler queries, faster reads, but more storage redundancy.
- **Snowflake Schema:** normalized dimensions broken into sub-tables, less redundancy, more flexibility, slightly more complex joins.

Telbs Eh adopts the Snowflake Schema because the location dimension naturally decomposes into Country → Governorate → City, and the weather condition dimension benefits from being normalized (condition → category). This avoids data redundancy when, for example, multiple cities share the same governorate.

3.4 Workflow Orchestration with Airflow

Apache Airflow is the de-facto open-source standard for orchestrating data pipelines. Its DAG (Directed Acyclic Graph) model fits ETL naturally: each step is a task, dependencies are explicit, and the scheduler handles retries, alerts, and backfilling. For Telbs Eh, Airflow schedules:

- A daily current-weather pull for all configured cities.
- A daily forecast pull for the next 5 days per city.
- A nightly load from OLTP into the Azure Data Warehouse.

3.5 Recommendation Systems Overview

Recommendation systems generally fall into three categories:

- **Content-based filtering** — uses item attributes to recommend similar items.
- **Collaborative filtering** — uses user-user or item-item similarity from interaction data.
- **Knowledge-based / rule-based systems** — use explicit domain rules to map inputs to recommendations.

Telbs Eh follows a knowledge-based / rule-based approach. Outfit recommendations are derived from explicit rules mapping weather attributes (temperature ranges, humidity, wind, precipitation, sky condition) to clothing categories. This is appropriate because: (a) the project

does not yet collect user interaction data, (b) the domain has well-known rules people already follow intuitively, and (c) results are explainable, which is a strong advantage for an academic deliverable.

3.6 Comparable Solutions

Several apps exist around the same idea (e.g. global outfit-suggestion services and weather-styling features in fashion apps). The differentiators of Telbs Eh are:

- Egypt-specific city coverage with reconciled Arabic/English naming.
- Open, transparent rule-based engine — explainable to academic reviewers.
- Full data engineering backbone (ETL + OLTP + DW + orchestration), not just a frontend layer.
- Cloud-deployable architecture on Azure with a real Snowflake Schema.

4. Requirements Gathering

Requirements were collected through team brainstorming sessions, informal interviews with potential users (peers and family members across different Egyptian governorates), and analysis of competing solutions. They are split into functional, non-functional, and data-specific requirements.

4.1 Stakeholders

Stakeholder	Interest in the System
End Users	Want quick, accurate outfit suggestions for the day based on their city's weather.
Project Team	Build, deploy, and maintain the full data pipeline and application.
Academic Supervisors	Evaluate the technical depth, documentation quality, and engineering rigor.
Data Analysts (future)	Use the warehouse for trend analysis (weather patterns, recommendation distribution).

4.2 Functional Requirements

The system shall support the following functional capabilities:

ID	Requirement	Description
FR-01	City Selection	The user shall be able to select their Egyptian city from a curated list.
FR-02	Current Weather Display	The system shall display the current weather for the chosen city: temperature, humidity, wind, condition.
FR-03	Forecast Display	The system shall display a multi-day forecast for the chosen city.
FR-04	Outfit Recommendation	Based on weather attributes, the system shall recommend an appropriate outfit category.
FR-05	Data Refresh	The system shall refresh weather data on a scheduled basis (via Airflow).
FR-06	Historical Storage	The system shall store weather and recommendation history in the Data Warehouse.

ID	Requirement	Description
FR-07	Statistical Reporting	The DW shall support analytical queries on weather trends and recommendation distribution.
FR-08	Data Quality Handling	The ETL shall reconcile inconsistent city names and fill missing coordinates from a manual mapping.

4.3 Non-Functional Requirements

ID	Quality Attribute	Requirement
NFR-01	Reliability	ETL jobs shall succeed for at least 99% of scheduled runs; failures shall trigger alerts.
NFR-02	Performance	A recommendation request shall return in under 2 seconds for any supported city.
NFR-03	Scalability	The architecture shall support adding new cities or attributes without redesigning the schema.
NFR-04	Maintainability	The codebase shall follow modular structure (separate extract, transform, load modules).
NFR-05	Data Quality	No row in OLTP shall have NULL critical fields (city, lat, lon, temperature).
NFR-06	Security	API keys shall be stored securely (environment variables / Azure Key Vault).
NFR-07	Portability	The pipeline shall be deployable to any cloud environment with minimal configuration changes.

4.4 Data Requirements

The following data attributes are required from the OpenWeather API and must be stored:

- City identifier, name (Arabic + English), country code.
- Latitude and longitude (validated, with manual fallback when missing).
- Current temperature (°C), feels-like, min/max.
- Humidity (%), wind speed and direction, atmospheric pressure.
- Weather condition code, main category (Clear / Clouds / Rain / etc), description.
- Sunrise / sunset timestamps.
- Forecast records for the next 5 days at 3-hour intervals.

- Timestamp of fetch (used for warehouse historization).

4.5 Use Cases (Summary)

UC-ID	Use Case	Brief Flow
UC-01	Get Outfit Suggestion	User selects city → app reads latest weather from OLTP → engine returns outfit category.
UC-02	View Forecast	User chooses a city → app shows next 5 days of weather + outfit hint per day.
UC-03	Refresh Weather Data	Airflow DAG runs on schedule → ETL pulls data → OLTP is updated.
UC-04	Load Warehouse	Nightly DAG → snapshot OLTP → load into Azure DW dimensions and fact tables.
UC-05	Run Statistical Query	Analyst queries DW for trends (e.g. distribution of recommendations per governorate).

4.6 Detailed Use Case Diagram

The following diagram captures the actors, primary use cases, and the relationships (associations, includes, and extends) of the Telbs Eh system. Three actors interact with the system: the End User (primary), the System Scheduler (Apache Airflow, acting as an automated time-triggered actor), and the Data Analyst (secondary).

Use Case Overview

25. End User: selects a city, views current weather, views the 5-day forecast, and receives an outfit recommendation.
26. System Scheduler (Airflow): automatically triggers the current weather ETL every 2 hours and the forecast ETL daily. Both runs include city-name reconciliation and DWH loading.
27. Data Analyst: runs statistical queries on the DWH and can export reports.
28. Key relationships: Get Outfit Recommendation always invokes the rule engine and logs the result. All ETL runs include the city reconciliation step. Forecast view is an optional extension of current weather view.

4.6.1 Actors Description

Actor	Type	Description
End User	Primary	A person living in or visiting Egypt who wants outfit recommendations based on the weather of their city.

Actor	Type	Description
System Scheduler (Airflow)	Secondary	An automated, time-triggered actor that initiates ETL DAGs at scheduled intervals.
Data Analyst	Secondary	Queries the analytical Data Warehouse to produce statistical insights and reports.

4.6.2 Detailed Use Case Specifications

Below are detailed specifications for the three most critical use cases of the system.

UC-04 — Get Outfit Recommendation

Use Case ID	UC-04
Name	Get Outfit Recommendation
Primary Actor	End User
Goal	Receive a clothing recommendation appropriate for the current weather in the selected city.
Preconditions	- The user has selected a valid Egyptian city. - OLTP database contains a fresh weather record for that city. - Outfit rules are loaded in the application layer.
Main Flow	- User opens the Telbs Eh application. - User selects a city from the dropdown list. - System fetches the latest weather snapshot from the OLTP store. - System invokes the recommendation engine (UC-05). - Engine evaluates rules: temperature range + condition + wind + humidity. - System returns the matched outfit category to the user. - System logs the recommendation event (UC-12) for analytics.
Alternate Flow	3a. If no fresh weather record exists, system displays a 'data unavailable' message. 5a. If multiple rules match, the engine returns the most specific match (e.g. rain rule overrides general temperature rule).
Postconditions	- User sees an outfit recommendation on screen. - A new row is inserted into recommendations_log.
Includes	UC-05 (Apply Weather → Outfit Rules), UC-12 (Log Recommendation)

UC-06 — Refresh Current Weather (ETL)

Use Case ID	UC-06
--------------------	-------

Name	Refresh Current Weather (ETL Pipeline)
Primary Actor	System Scheduler (Apache Airflow)
Goal	Keep the OLTP database up to date with the latest weather observations for all configured Egyptian cities.
Preconditions	• Airflow DAG schedule fires (hourly). • OpenWeather API is reachable and the API key is valid. • The master city list is available.
Main Flow	1. Airflow triggers the dag_weather_current DAG. 2. Extract task reads the master city list. 3. For each city, the system calls the OpenWeather current weather endpoint. 4. Transform task parses JSON, applies the city-name mapping (UC-08). 5. Transform task validates types and rejects malformed rows. 6. Load task upserts records into the weather_current OLTP table. 7. Airflow marks the DAG run as successful and logs metrics.
Alternate Flow	3a. If the API returns an error or rate limit, the task retries up to 3 times with exponential backoff. 5a. Rejected rows are written to a quarantine table for manual review. 6a. If the load fails, the entire DAG run is marked as failed and an alert is dispatched.
Postconditions	• weather_current table reflects the latest weather data. • Ingestion timestamp is set on each row for downstream warehouse loading.
Includes	UC-08 (Reconcile City Names & Coordinates)

UC-08 — Reconcile City Names & Coordinates

Use Case ID	UC-08
Name	Reconcile City Names & Coordinates
Primary Actor	System Scheduler (Airflow) — invoked from UC-06 / UC-07
Goal	Resolve mismatches between OpenWeather API names and the canonical Egyptian city names, and fill missing latitude/longitude values from the manual mapping file.
Preconditions	• Three reference files exist: master cities (API), manual lat/lon, not-found list. • The ETL is currently running.

Main Flow	1. Load master city list (API-derived) into a Pandas DataFrame. 2. Load manual lat/lon file into a second DataFrame. 3. Apply name-mapping dictionary (e.g. 'The Lake' → 'Beheira / البحيرة'). 4. Merge master DataFrame with the manual DataFrame on city_id / canonical_name. 5. Drop duplicates and fill nulls in lat/lon from the manual source. 6. Output a unified, validated DataFrame ready for downstream loading.
Alternate Flow	3a. If a new API name appears that has no mapping entry, the row is logged to lat_lon_not_found.xlsx for manual addition. 5a. If after merge any lat/lon is still NULL, the city is excluded from the current ETL run and reported.
Postconditions	• A clean, complete DataFrame of cities with valid coordinates is available to the calling ETL. • Any unresolved cities are flagged for human review.

4.6.3 Use Case Relationships Summary

From	Relationship	To	Meaning
UC-04 (Get Recommendation)	«include»	UC-05 (Apply Rules)	Recommendation always invokes the rule engine.
UC-04	«include»	UC-12 (Log Recommendation)	Every recommendation is logged for analytics.
UC-02 (View Current Weather)	«extends»	UC-03 (View Forecast)	Forecast view is an optional extension of current weather view.
UC-06 (Refresh Current)	«include»	UC-08 (Reconcile Names)	Every ETL run reconciles names and coordinates.
UC-07 (Refresh Forecast)	«include»	UC-08 (Reconcile Names)	Forecast ETL also reuses the reconciliation logic.
UC-10 (Run Statistical Analysis)	«extends»	UC-11 (Export Reports)	Analysts can optionally export their results.

5. System Analysis & Design

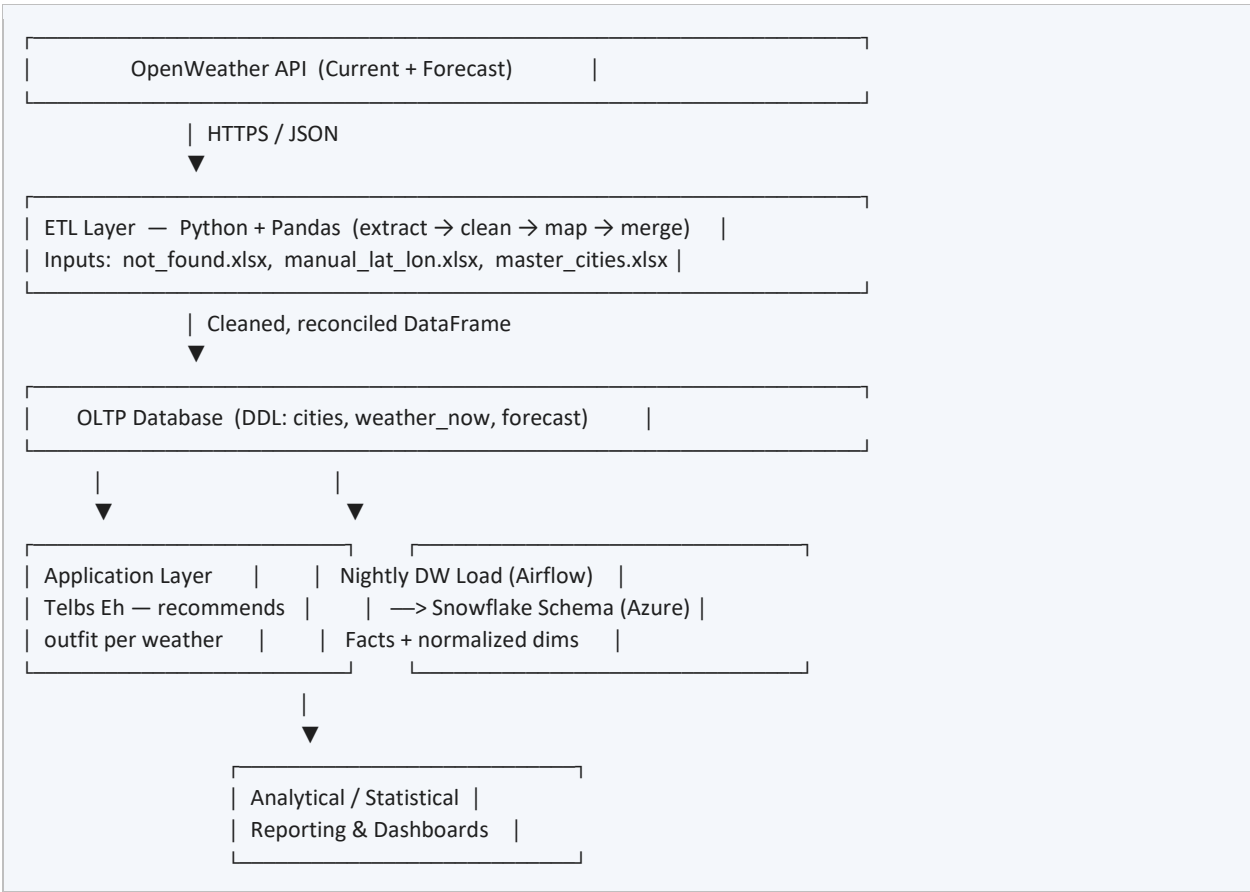
This section explains the system's architecture, data model at each layer, the ETL pipeline strategy used to reconcile real-world data issues, the recommendation engine, and the analytical Snowflake Schema deployed on Azure.

5.1 High-Level Architecture

Telbs Eh is composed of five layers, each with a distinct responsibility:

- **1. Data Source Layer** — OpenWeather public REST API (current + forecast).
- **2. Ingestion / ETL Layer** — Python + Pandas pipelines orchestrated by Airflow.
- **3. OLTP Layer** — Operational SQL store powering the application in real time.
- **4. Application Layer** — Recommendation engine that maps weather attributes to outfit categories.
- **5. Analytical Layer** — Snowflake Schema Data Warehouse on Microsoft Azure.

5.1.1 Architecture Diagram (Logical Flow)



5.2 Data Acquisition Strategy

The cities of Egypt were collected manually as the master reference list, since the OpenWeather geocoding endpoint does not always return a complete or correctly named set of Egyptian regions. For each city in this reference list, the team queried the API to retrieve latitude, longitude, and weather data.

5.2.1 Two Real-World Data Issues Encountered

Issue #1 — Missing coordinates: Some Egyptian regions did not return any latitude/longitude from the API. These were exported into a *not-found file* and resolved manually using authoritative geographic sources (e.g. publicly known coordinates of governorate centers).

Issue #2 — Naming mismatches: The API sometimes returned literal English translations of Arabic place names rather than their conventional transliteration. For example, `Beheira` (Beheira) was returned as *"The Lake"*. These were resolved by building a mapping inside the Python ETL between API names and internal canonical names.

5.2.2 Three-File Resolution Approach

The reconciliation produces three working files, each with a clear role:

System Architecture — Data Flow

29. 1. OpenWeather API — data source providing current weather and 5-day forecast via HTTPS/JSON.
30. 2. ETL Layer (Python + Pandas) — extracts from API, cleans data, reconciles city names and coordinates, produces a unified DataFrame.
31. 3. OLTP Database (MySQL: telbs_eh_final_proj) — operational snapshot store with OLTP_current_weather and OLTP_forecast_weather tables. Truncated and reloaded on each run.
32. 4. Application Layer (Streamlit) — reads from OLTP and displays real-time weather, forecast, and outfit recommendations to the user.
33. 5. DWH Load (Airflow DAG) — before each OLTP truncate, data is saved permanently to the DWH (fact_weather_observation, fact_weather_forecast).
34. 6. Data Warehouse (MySQL: telbs_eh_dwh) — historical store with dim_city, dim_date, fact_weather_observation, and fact_weather_forecast tables. Never truncated.

Inside the ETL, the manual file is merged with the master file using Pandas' merge operation, producing a single, complete DataFrame of cities + coordinates with no missing critical fields. This DataFrame is then used as the authoritative city dimension for the rest of the pipeline.

5.3 ETL Pipeline Design

The ETL pipeline is implemented in Python and follows a clear three-step structure:

5.3.1 Extract

- Read the master city list (manual + API-derived) from Excel.
- Iterate over each city and call OpenWeather: current weather endpoint.
- Call the forecast endpoint for the same city to retrieve future predictions.
- Persist raw JSON for traceability (debugging, audit).

5.3.2 Transform

- Parse JSON responses into Pandas DataFrames.
- Apply the API-name → canonical-name mapping (fixes the Beheira / The Lake type issues).
- Coerce data types (numerics for temperature, datetime for timestamps, etc.).
- Reject rows missing critical fields and log them for review.
- Derive helper columns: feels-like bucket, day/night flag, weather category.

5.3.3 Load

- Insert / upsert rows into the OLTP tables defined by the DDL.
- Tag each row with an ingestion timestamp for warehouse historization.
- Trigger downstream tasks (recommendation cache refresh, DW load).

5.4 OLTP Schema (DDL Layer)

The OLTP database is normalized to support the application layer. Core tables:

Table	Purpose / Key Columns
cities	city_id (PK), name_ar, name_en, governorate, latitude, longitude.
weather_current	id (PK), city_id (FK), temperature, feels_like, humidity, wind, condition_code, fetched_at.
weather_forecast	id (PK), city_id (FK), forecast_for_ts, temperature, condition_code, humidity, wind, fetched_at.
weather_conditions	condition_code (PK), main, description, icon — reference table from the API.
outfit_categories	outfit_id (PK), name, description, suitable_temperature_min, suitable_temperature_max.

Table	Purpose / Key Columns
recommendations_log	id (PK), city_id (FK), outfit_id (FK), recommended_at, weather_snapshot_id (FK).

5.5 Application Layer — Recommendation Engine

The application reads the latest weather snapshot for the user's city from the OLTP store, evaluates a rule set, and returns one or more outfit categories. The rules are weather-attribute driven:

Temperature Range	Condition	Recommended Outfit (example)
$\geq 30^{\circ}\text{C}$	Clear / Sunny	Light cotton t-shirt, shorts, sunglasses, hat.
$22 - 29^{\circ}\text{C}$	Clear / Clouds	T-shirt, light pants, comfortable shoes.
$15 - 21^{\circ}\text{C}$	Clouds	Long-sleeve shirt, jeans, light jacket.
$8 - 14^{\circ}\text{C}$	Clouds / Wind	Sweater, jacket, jeans, closed shoes.
$< 8^{\circ}\text{C}$	Cold	Heavy coat, scarf, layered clothing, boots.
Any	Rain	Add waterproof jacket / umbrella regardless of temperature bucket.
Any	High Wind	Add windbreaker; avoid loose accessories.

The rule layer is intentionally explicit and rule-based so the system's recommendations are explainable. The engine logs every recommendation into the recommendations_log table, which feeds the warehouse for analytical reporting.

5.6 Data Warehouse — Snowflake Schema (Azure)

The analytical layer is modeled as a Snowflake Schema on Microsoft Azure. The fact table captures recommendation events, surrounded by normalized dimensions for location, time, weather conditions, and outfit categories.

5.6.1 Fact Table

- **fact_recommendation** — one row per recommendation event. Columns: recommendation_id, date_id, time_id, city_id, condition_id, outfit_id, temperature, humidity, wind_speed.

5.6.2 Dimension Tables (Snowflaked)

Dimension	Normalization (Snowflake style)
dim_date	date_id, full_date, day, month, quarter, year, weekday, is_weekend.
dim_time	time_id, hour, minute, period (morning / afternoon / evening / night).
dim_city → dim_governorate → dim_country	Cities link to governorates; governorates link to country (Egypt). Classic snowflake decomposition.
dim_condition → dim_condition_main	Specific weather condition (e.g. light rain) → main category (Rain). Allows roll-up.
dim_outfit → dim_outfit_category	Specific outfit → outfit category (light, casual, warm, heavy, rain-gear). Enables aggregation.

5.6.3 Sample Analytical Questions Answered by the DW

- How does the distribution of recommended outfits change between summer and winter per governorate?
- What is the average temperature trend per month across Egyptian governorates?
- Which weather conditions most frequently trigger rain-gear recommendations?
- Which cities have the highest variance between forecast and actual readings?

5.7 Orchestration with Apache Airflow

Airflow runs three primary DAGs:

- **current_weather_dag** — runs every 2 hours: fetches current weather, saves to DWH, then refreshes the OLTP snapshot.
- **forecast_weather_dag** — runs daily at midnight: fetches 5-day forecast, saves to DWH, then refreshes the OLTP forecast snapshot.
- DWH loading is handled inside each DAG run before truncating the OLTP tables.

Each DAG includes retry logic, failure alerts, and SLA monitoring.

5.8 Deployment on Microsoft Azure

- OLTP and DWH databases hosted on MySQL running locally on Windows, accessible from WSL via a static host IP.
- Data Warehouse modeled with fact and dimension tables: dim_city, dim_date, fact_weather_observation, fact_weather_forecast.

- Apache Airflow runs inside a Docker container using docker-compose, with DAG files mounted via a local volume.
- API keys and DB credentials managed via environment variables and Docker configuration.
- Streamlit dashboard runs locally and connects to the MySQL OLTP database to serve real-time weather and outfit recommendations.

5.9 Current Status & Next Steps

The project is fully implemented and operational. The following has been completed:

- OpenWeather API integration (current + forecast).
- Manual curation of Egyptian regions with reconciled lat/lon.
- Python + Pandas ETL with mapping of inconsistent names.
- OLTP DDL designed and populated.
- Three reconciliation files produced and merged into a unified DataFrame.
- Streamlit dashboard fully built — displays current weather, 5-day forecast, outfit recommendations, and historical readings.

Completed deliverables:

35. Airflow DAGs running on schedule inside Docker (current_weather_dag every 2 hours, forecast_weather_dag daily).
36. DWH fully loaded with fact_weather_observation and fact_weather_forecast populated on every DAG run.
37. Streamlit dashboard live — real-time weather, forecast, outfit recommendations, and historical readings.
38. End-to-end pipeline tested and verified: API → ETL → OLTP → DWH → Dashboard.

6. Conclusion

Telbs Eh (تلبس ايه؟) demonstrates how a familiar, everyday question can be turned into a well-structured data engineering project. The system covers the full lifecycle of a data-intensive application: ingestion from a public API, real-world data reconciliation, transactional storage for live use, an explainable recommendation engine, analytical warehousing on the cloud, and orchestrated automation.

Beyond the application itself, the project showcases practical decisions taken to handle imperfect real-world data — missing coordinates, inconsistent naming between Arabic and English, and the need for a mapping layer between API responses and internal canonical references. These are exactly the kinds of issues that distinguish an academic exercise from an applied data engineering effort.

With the planned Snowflake Schema on Azure and the Apache Airflow orchestration in place, Telbs Eh is positioned not only as a recommendation app but also as a foundation for ongoing weather and lifestyle analytics across Egypt.